

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего
образования
«Национальный исследовательский технологический университет «МИСИС»
(НИТУ «МИСИС»)

ИНСТИТУТ
Институт компьютерных наук
КАФЕДРА
инфокоммуникационных технологий
НАПРАВЛЕНИЕ
09.03.02 — Информационные системы и технологии

ДОМАШНЕЕ ЗАДАНИЕ № 1

по дисциплине «Оптимизация клиент-серверных приложений»
на тему: «Разработка сервиса по индивидуальному варианту и измерение исходных
показателей производительности»
Вариант 3

Студент группы БИСТ-24-ПО-1 _____ Невокшенов Е. А.

Преподаватель _____ Жердев А. А., к.т.н.,
доцент

Москва, 2026

СОДЕРЖАНИЕ

ЗАДАНИЕ И ВАРИАНТ.....	3
ВВЕДЕНИЕ.....	4
1 Сервис, стенд и данные.....	5
1.1 Сервис и контракт.....	5
1.2 Модульные тесты.....	8
1.3 Условия испытания.....	8
1.4 Наполнение базы данных.....	10
2 Измерения.....	11
2.1 Время ответа операций.....	11
2.2 Разложение времени между приложением и базой данных.....	12
3 Гипотезы об узких местах.....	13
ЗАКЛЮЧЕНИЕ.....	15

ЗАДАНИЕ И ВАРИАНТ

Таблица 1 – Вариант и стенд

Параметр	Значение
Вариант и предметная область	3 — запись на приём к специалисту
Префикс	evgenii_nevokshenov
Язык и каркас	Python 3.12.10, FastAPI 0.115.12
Доступ к данным	psycopg 3.2.9, явные SQL-запросы
СУБД	PostgreSQL 16.15 on aarch64-unknown-linux-musl
Клиентская часть	HTML, CSS, JavaScript; выдаётся сервером
Развёртывание	Docker Compose; PostgreSQL и приложение
Репозиторий	Публикация запланирована отдельно; ссылка добавляется перед сдачей.

Сервис хранит расписание специалистов и показывает свободные слоты для выбранной услуги. Бронирование отклоняется при пересечении с активной записью, неподходящей длительности или недоступности услуги у специалиста. Отмена сохраняет историю и освобождает время. Сводка показывает загрузку специалистов за выбранный период и долю отменённых записей.

ВВЕДЕНИЕ

Исходные показатели производительности нужны для последующего доказательства эффекта оптимизации. Оценка по субъективному впечатлению от интерфейса не позволяет отличить улучшение кода от изменения объёма данных, прогрева процесса или условий запуска. Поэтому до оптимизации фиксируются технологии, данные, сценарии и условия испытаний, а время измеряется серией повторов.

В отличие от лабораторного учебного стенда объектом исследования является собственный сервис записи на приём. Сначала подтверждается правильность его поведения, затем измеряется зависимость времени ответа от объёма и устанавливаются направления дальнейшего исследования.

Цель работы — разработать сервис по варианту 3 и получить воспроизводимые исходные показатели его производительности на двух объёмах данных.

Для достижения цели необходимо решить следующие задачи:

1. Реализовать сервис записи на приём с авторизацией и клиентским интерфейсом.
2. Описать контракт программного интерфейса.
3. Проверить бизнес-правила и операции интерфейса тестами.
4. Подготовить воспроизводимые малое и рабочее наполнения PostgreSQL.
5. Измерить время ответа всех операций на обоих объёмах.
6. Разложить серверное время операций между обращениями к БД и приложением.
7. Сформулировать гипотезы об узких местах по полученным числам.
8. Проверить очевидные ошибки реализации и зафиксировать результат проверки.

1 Сервис, стенд и данные

1.1 Сервис и контракт

Реализованы вход, список записей с фильтрами и пагинацией, карточка, поиск слотов с бронированием и сводка. Авторизация использует серверную сессию в HttpOnly cookie, действующую восемь часов. Демо-пользователь demo выполняет все сценарии; пароль хранится в виде PBKDF2-хеша. Пересекающиеся бронирования одного специалиста сериализуются блокировкой строки специалиста в транзакции.

Основные сущности: специалист, услуга, слот и запись. Поддерживаемые услуги заданы массивом идентификаторов в специалисте. Пользователи и сессии являются техническими таблицами. При отмене запись получает статус cancelled, а последующее бронирование проверяет только активные записи.

Таблица 2 – Операции программного интерфейса

Метод и путь /api	Параметры	Ответ	Ошибки
POST /login	username, password	id, username; cookie	401, 422
POST /logout	cookie	ok=true	401
GET /me	cookie	id, username	401
GET /services	—	items: услуги	401
GET /specialists	—	items: специалисты	401
GET /appointments	page, size, status, specialist_id, from_at, to_at	items, total, page, size	401, 422
GET /appointments/{id}	id	карточка со связями	401, 404, 422
GET /slots	service_id, specialist_id, период, page, size	items, total, page, size	401, 404, 422
POST /appointments	slot_id, service_id, client_name, start_at?	201; карточка	401, 404, 409, 422
POST /appointments/{id}/cancel	id	карточка; cancelled	401, 404, 409, 422

Метод и путь /api	Параметры	Ответ	Ошибки
GET /summary	from_at, to_at	показатели специалисты и	401, 422

Полный контракт с полями карточки и сводки находится в docs/API.md. Размер страницы составляет от 1 до 100 строк. Даты содержат часовой пояс; интервал периода полуоткрытый. Поиск слотов ограничен 31 днём. Отсутствие авторизации даёт 401, ошибки параметров — 422, отсутствующая запись — 404, конфликт бронирования или повторная отмена — 409. Кеширование не используется.

The screenshot shows the 'ПРИЁМ' (Reception) web application. At the top, there's a header with the logo 'ПРИЁМ' and the tagline 'Сервис записи к специалисту' (Specialist booking service). On the right, a user is logged in as 'evgenii_nevokshenov' with a 'Выйти' (Logout) button. The main section is titled 'РАСПИСАНИЕ И ЗАПИСИ' (Schedule and Bookings) and 'Выберите удобное время' (Choose a convenient time). Below this, there are tabs for 'Все записи' (All bookings), 'Новая запись' (New booking), and 'Сводка' (Summary). The 'Новая запись' tab is active. The search filters include 'Услуга' (Service) set to 'Консультация 1 · 20 мин', 'Специалист' (Specialist) set to 'Все специалисты' (All specialists), 'С даты' (From date) set to '01.10.2026', and 'До даты включительно' (To date inclusive) set to '07.10.2026'. A 'Найти слоты' (Find slots) button is present. The results section, titled 'Свободное время' (Free time), shows a grid of available slots. Each slot card displays the specialist's name (e.g., 'Иванов · специалист 1' or 'Попова · специалист 6'), the date and time (e.g., '01.10.2026, 09:00'), and a 'Записаться →' (Book →) button. A status 'Свободных слотов: 260' (Free slots: 260) is shown in the top right of the results area.

Рисунок 1 – Поиск свободных слотов и переход к бронированию

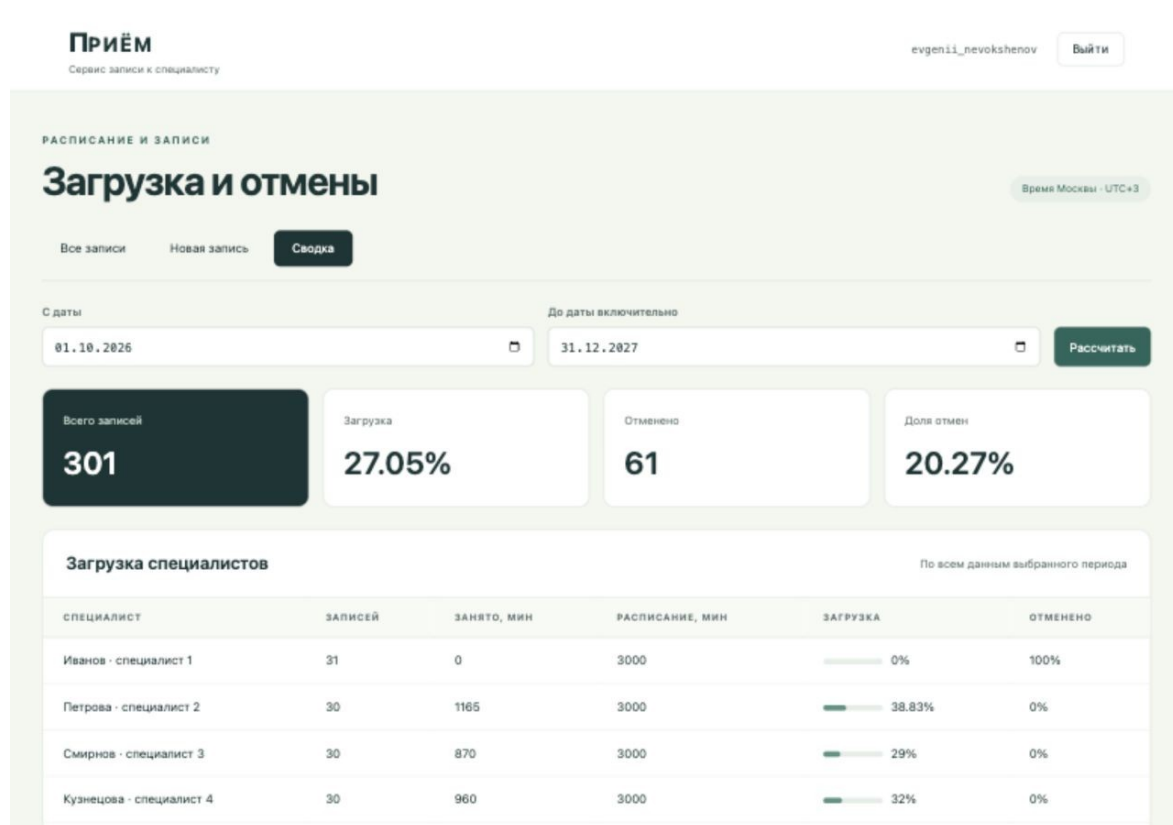


Рисунок 2 – Сводка на малом наборе после проверки интерфейса

Скриншоты получены при ручной проверке интерфейса. В основной базе дополнительно создана и отменена одна проверочная запись: на рисунке 2 поэтому 301 запись. Измерения выполнялись независимо, в отдельной базе evgenii_nevokshenov_measure.

1.2 Модульные тесты

Проверяются бизнес-правила интервалов и длительности, доступность услуги, расчёт долей, авторизация, поля ответов, фильтры, пагинация, отмена и конкурентное бронирование. Дополнительно проверены повторяемость наполнения и обработка ошибок измерителя. API-тесты используют реальную отдельную PostgreSQL, а чистые правила не обращаются к БД.

Команда запуска: `bash scripts/control.sh tests`. Итоговый прогон: 33 passed. Тестовая БД `evgenii_nevokshenov_test` изолирована от данных приложения.

Листинг 1 – Примеры тестов бизнес-правила и контракта

```
def test_adjacent_intervals_are_allowed():
    assert not overlaps(START, START+timedelta(hours=1),
                        START+timedelta(hours=1),
                        START+timedelta(hours=2))

def test_authorization_is_required(client):
    for path in ['/api/me', '/api/appointments',
                '/api/services', '/api/specialists',
                '/api/slots?service_id=1', '/api/summary']:
        assert client.get(path).status_code == 401
```

Тест конкурентного бронирования выполняет два одновременных запроса и получает ровно один ответ 201 и один 409. Проверена также обрезка занятых минут границами периода; пустая сводка возвращает нулевые доли.

1.3 Условия испытания

Таблица 3 – Условия испытания

Что	Значение
Машина	Apple M3 Pro; 18 ГБ RAM; macOS 26.6.2
Среда выполнения	Docker 27.5.1, Linux ARM64; 11 CPU, около 7,65 ГиБ RAM для Docker; один процесс uvicorn
Учётная запись	demo; одинаковая для всех запросов
Повторы	30 измеряемых запросов для каждой операции и каждого объёма
Прогрев	5 успешных запросов перед каждой серией; исключены из расчётов

Что	Значение
Инструмент	httpx 0.28.1, perf_counter; последовательный HTTP по loopback внутри контейнера
Периоды	Слоты: 1–8 октября 2026; список и сводка: 1 октября 2026 — 1 января 2028
Процентили	p50 — медиана; p95 — ближайший ранг $\text{ceil}(0,95 \times N)$; максимум по 30 запросам
Прочие процессы	На протяжении серий запущены также portal-06-api, portal-06-weather и portal-06-db; нагрузка на них не создавалась. Результаты относятся к общему локальному Docker-стенду.

Оба объёма исследовались одной версией кода на одном стенде. Замеры не являются нагрузочным испытанием: конкурентность равна единице, кроме отдельного теста корректности. Точные версии зависимостей записаны в requirements.lock.txt и results/measurements.json. Данные и показатели получены 4 октября 2026 года.

1.4 Наполнение базы данных

Наполнение выполняет команда `python -m scripts.seed --size small|work --reset`. Для обычного сервиса она запускается через `docker compose exec app`. Скрипт измерений вызывает тот же генератор в отдельной исследовательской БД и дважды формирует каждый объём; контрольные суммы всех четырёх таблиц совпали.

Таблица 4 – Объём исходного наполнения

Таблица	Малое	Рабочее	Рост
specialists	10	200	20 раз
services	10	30	3 раз
slots	500	200000	400 раз
appointments	300	60000	200 раз

Генерация имеет фиксированную дату и порядок; активные записи не пересекаются. Исходная доля отменённых записей равна 20%. Фактические количества получены запросами `count(*)` к PostgreSQL, а контрольные суммы — из упорядоченного представления строк. Личные имена и сведения в базе синтетические.

Для измерения операций записи к каждому набору добавлено по 70 слотов и подготовлено по 35 записей для отмены. Прогрев создания добавил 5 записей, измеряемая серия — ещё 30; итого добавлено 70 записей, включая подготовку отмены. В конце малый набор содержит 570 слотов и 370 записей, рабочий — 200070 слотов и 60070 записей. Чтения измерены до серий создания и отмены, но после подготовки 35 записей отмены. Эти одинаковые добавки зафиксированы отдельно и не скрыты в объёме исходного наполнения.

2 Измерения

2.1 Время ответа операций

Таблица показывает полное время получения HTTP-ответа в миллисекундах. Все 11 операций контракта измерены на обоих объёмах. Исходные 30 значений каждой серии сохранены в results/measurements.json; значения прогресса в таблицу не включены.

Таблица 5 – Время ответа на двух объёмах данных в мс

Операция	p50 мал.	p95 мал.	max мал.	p50 раб.	p95 раб.	max раб.	Рост p50
Вход	25,78	28,08	28,30	26,53	30,23	30,72	1,03
Выход	4,63	5,43	5,78	4,64	6,39	7,30	1,00
Пользователь	4,27	4,48	4,78	4,73	5,37	7,14	1,11
Услуги	4,68	5,26	5,93	4,60	4,78	4,87	0,98
Специалисты	4,63	5,89	6,05	6,03	6,22	6,23	1,30
Список записей	6,02	6,32	6,44	34,45	37,22	37,37	5,72
Карточка	5,36	6,40	6,78	5,03	5,17	5,40	0,94
Свободные слоты	6,90	7,08	7,17	138,31	142,89	143,22	20,05
Сводка	8,03	8,96	9,69	498,70	503,70	512,57	62,08
Бронирование	8,50	13,75	21,68	9,17	10,30	10,54	1,08
Отмена	7,08	8,74	8,99	5,66	5,96	6,48	0,80

Список замедлился в 5,72 раза, свободные слоты — в 20,05, сводка — в 62,08. Это главные кандидаты на дальнейшее исследование. Объём записей вырос в 200 раз, слотов — в 400 раз; ни одна из этих операций не выросла быстрее исходных данных, однако их абсолютные затраты существенно увеличились.

Вход, выход, текущий пользователь, услуги, карточка и отмена остаются близкими по времени и исключены из дальнейшего разбора зависимости от объёма. Список специалистов вырос в 1,30 раза, создание — в 1,08; они включены в разложение как второстепенные случаи. Время входа

определяется прежде всего проверкой PBKDF2, а не числом записей на приём.

2.2 Разложение времени между приложением и базой данных

На рабочем объёме использованы серверные метрики Server-Timing. SQL-время измеряется вокруг `execute` и `fetchall`: включает ожидание PostgreSQL, передачу строк и их декодирование драйвером. Оно не является чистым временем исполнения внутри PostgreSQL. Время кода — остаток серверного времени после SQL; сюда входят открытие соединения, транзакционное завершение, бизнес-логика и подготовка ответа. Внешняя доставка HTTP измерена отдельно в таблице 5.

Для аддитивности разложения таблица 6 содержит арифметические средние по одним и тем же 30 запросам: серверное время равно сумме SQL и остатка. Медианы отдельных частей не складывались. Число SQL-запросов в каждой серии постоянно и включает авторизацию.

Таблица 6 – Куда уходит серверное время операции

Операция	Всего, мс	SQL, мс	Код, мс	Запросов	Вывод
Список записей	34,07	29,65	4,43	3	БД
Свободные слоты	138,47	134,24	4,22	4	БД
Сводка	497,28	236,62	260,66	4	БД и код
Специалисты	5,61	1,11	4,50	2	Код
Бронирование	8,67	5,03	3,64	7	БД

У списка и поиска слотов доминируют обращения к БД. Сводка тратит значимое время на обе части: большие наборы строк получают из БД и затем обрабатывают в Python. У специалистов большая часть небольшого времени остаётся вне SQL, в том числе при сериализации ответа. Семь запросов бронирования имеют фиксированные назначения: авторизация, слот, блокировка специалиста, услуга, конфликт, вставка, карточка. Запросов в цикле по строкам ответа нет; число запросов не растёт с размером базы.

3 Гипотезы об узких местах

Гипотеза 1. Список записей.

1. Операция: GET /api/appointments.

2. Основание: HTTP p50 6,02 → 34,45 мс; p95 на рабочем объёме 37,22 мс. Медиана серверного времени 33,71 мс, SQL 29,48 мс, остаток 4,38 мс; 3 запроса на операцию.

3. Предполагаемая причина: COUNT(*) и сортировка списка без дополнительных индексов просматривают существенно больше строк на рабочем объёме. Фиксированные три SQL-запроса исключают объяснение ростом их количества. Проверка гипотезы в следующих заданиях: EXPLAIN (ANALYZE, BUFFERS) и отдельное исследование подсчёта и выборки страницы.

Гипотеза 2. Свободные слоты.

1. Операция: GET /api/slots.

2. Основание: HTTP p50 6,90 → 138,31 мс; p95 на рабочем объёме 142,89 мс. Медиана серверного времени 137,62 мс, SQL 133,55 мс, остаток 4,07 мс; 4 запроса на операцию.

3. Предполагаемая причина: Поиск выполняет два соединения расписания с активными записями — для количества и страницы — и проверяет пересечения. При большом расписании и отсутствии дополнительных индексов стоимость фильтрации и соединения растёт. Проверка: планы обоих запросов и статистика прочитанных строк; затем индекс или изменение запроса с доказательством эффекта.

Гипотеза 3. Сводка.

1. Операция: GET /api/summary.

2. Основание: HTTP p50 8,03 → 498,70 мс; p95 на рабочем объёме 503,70 мс. Медиана серверного времени 497,94 мс, SQL 236,73 мс, остаток 260,47 мс; 4 запроса на операцию.

3. Предполагаемая причина: Приложение получает все подходящие слоты и записи, объединяет интервалы и агрегирует показатели в Python. Существенны и передача/декодирование строк, и обработка больших коллекций. Проверка: профиль Python, измерение размера ответа БД и сравнение с агрегацией на стороне PostgreSQL без изменения формул сводки.

При проверке исходной схемы обнаружен дополнительный индекс `username`, автоматически созданный ограничением `UNIQUE`. Он удалён, поскольку в ДЗ1 допустимы только индексы ключей; отдельный тест проверяет отсутствие вторичных индексов. После изменения заново сняты все серии на обоих объёмах. Данные до исправления сохранены отдельно в `results/measurements-before-index-fix.json`. Изменение касается технической таблицы одного демо-пользователя, а не предметных запросов. Медиана входа на рабочем объёме до исправления составляла 27.24 мс, после — 26.53 мс; это различие в повторных сериях не доказывает ускорения. Все итоговые таблицы используют только повторный прогон.

Очевидные ошибки. В итоговой версии ошибок контракта, пропущенной пагинации или повторного одинакового SQL-запроса не обнаружено. Это подтверждено 33 тестами, ручным выполнением входа, поиска, записи, просмотра и отмены, а также постоянным числом SQL-запросов. Подсчёт общего количества и выборка страницы являются разными запросами, а не случайным дублем. Все представленные серии относятся к одной проверенной версии бизнес-логики. Сохраняющиеся затраты являются предметом дальнейшего профилирования и оптимизации, а не основанием намеренно ухудшать код.

ЗАКЛЮЧЕНИЕ

Цель работы достигнута: реализован сервис записи на приём по варианту 3 и получены исходные показатели на двух воспроизводимых объёмах данных.

Для этого решены следующие задачи:

1. Реализован сервис с авторизацией, списком, карточкой, поиском слотов, бронированием, отменой и сводкой.
2. Описан контракт 11 операций программного интерфейса.
3. Выполнено 33 успешных тестов, включая конкурентное бронирование.
4. Подготовлены наборы с 300 и 60000 исходных записей, 500 и 200000 слотов; подтверждены повторные контрольные суммы.
5. Измерены 11 операций на двух объёмах: по 30 запросов после пяти прогревочных повторов, рассчитаны p50, p95 и максимум.
6. Разложено серверное время пяти операций; зафиксированы SQL-время, остаток и количество запросов.
7. Сформулированы три гипотезы по результатам измерений.
8. Проверены очевидные ошибки; нарушений контракта и лишних запросов в итоговой версии не выявлено.

Основные затраты обнаружены в списке, поиске свободных слотов и сводке. На рабочем объёме их HTTP-медианы составили соответственно 34,45, 138,31 и 498,70 мс. Для списка и слотов преобладают обращения к БД; для сводки значимы также обработка и агрегация в Python. Результаты дают исходную точку для нагрузочного исследования и последующей оптимизации при сохранении контракта.